PHP POO — Classes, Objets & Encapsulation

Semaine 3 · Mardi · 1h · C6 — Composants d'accès aux données | C7 — Back-end

1. Concepts clés

1.1 Rappel : classe, objet, $this

Depuis lundi vous savez créer une classe simple. Aujourd'hui on monte d'un cran : on contrôle qui peut lire et modifier les attributs.

Concept	Définition — Syntaxe PHP
class	Mot-clé pour déclarer une classe. → class NomClasse { }
new	Crée un objet (instance) de la classe. → $obj = new Produit(...);
$this	Référence l'objet courant à l'intérieur de la classe. → $this->nom
__construct()	Constructeur : s'exécute automatiquement lors du new. Initialise les attributs.
->	Opérateur d'accès : appelle une méthode ou lit un attribut public sur un objet.

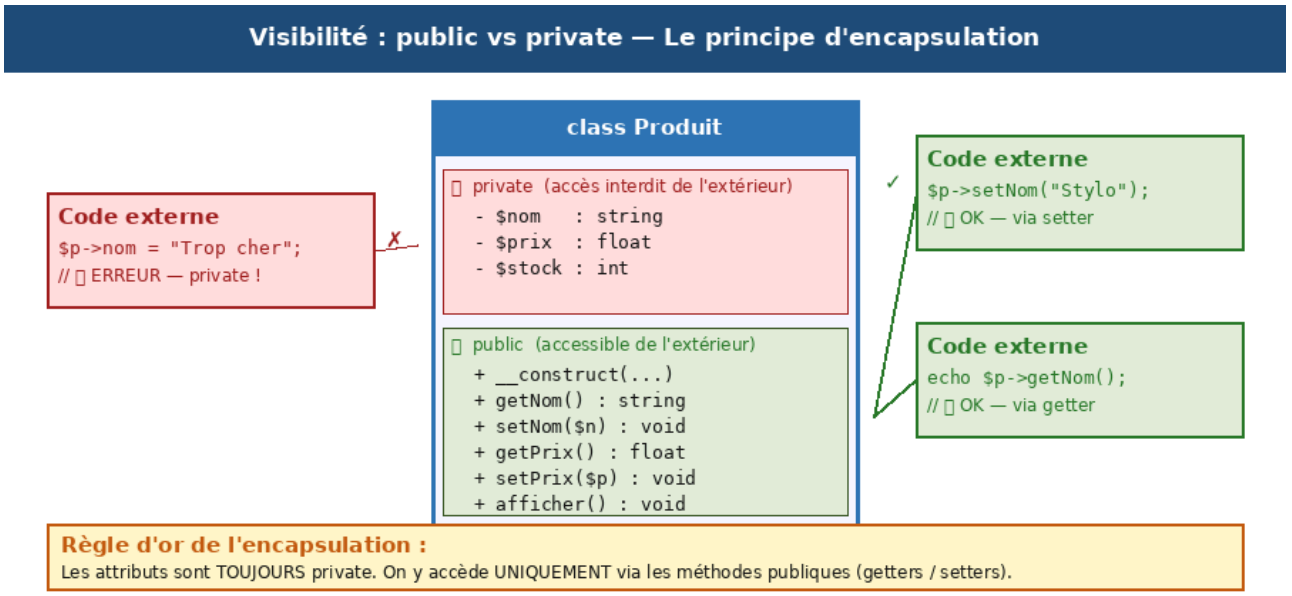
1.2 Visibilité : public / private / protected

Modificateur	Accès autorisé
public	Partout : depuis la classe, les sous-classes et le code externe
private	Uniquement dans la classe elle-même — le code externe ne peut pas y toucher
protected	Dans la classe ET dans ses sous-classes (héritage) — vu mercredi

Règle d'or : les attributs sont TOUJOURS private. Les méthodes sont public (sauf utilitaires internes).

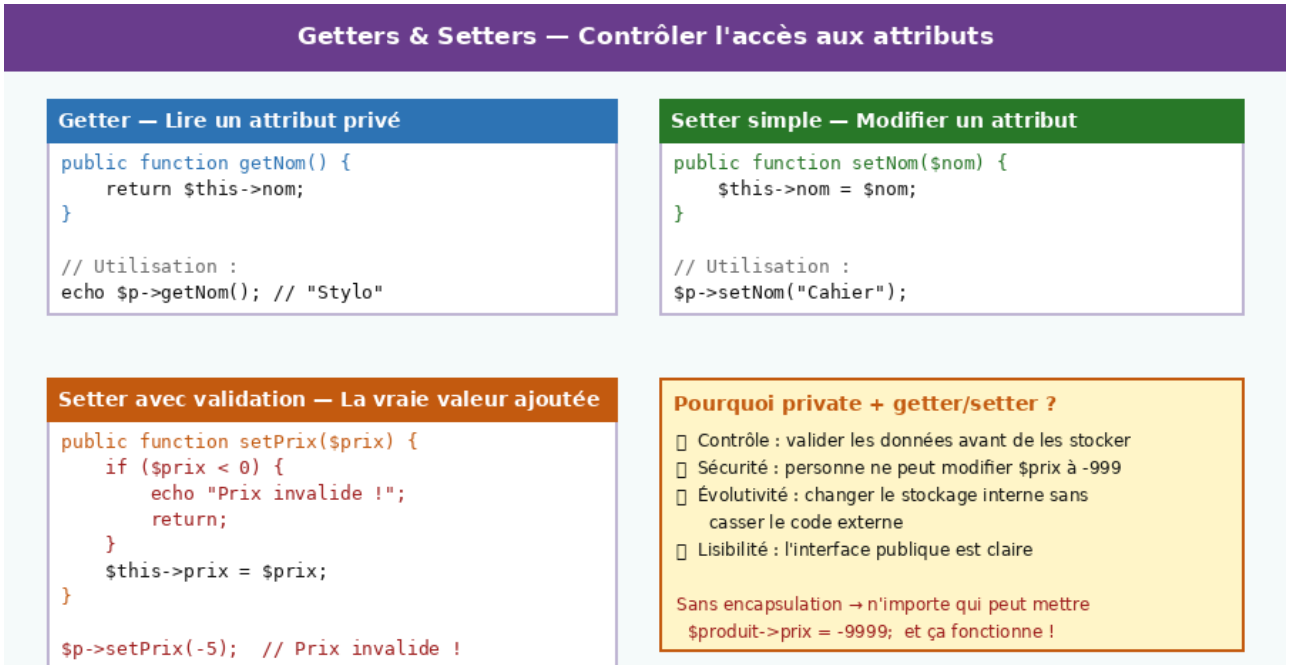
1.3 Encapsulation — Visualisation

L'encapsulation protège les attributs en les rendant private et expose uniquement des méthodes publiques contrôlées pour y accéder.



1.4 Getters & Setters

Les getters lisent un attribut privé. Les setters le modifient — avec la possibilité d'y ajouter une validation.



Méthode	Convention de nommage — Exemple
Getter	get + NomAttribut() → getNom(), getPrix(), getStock()
Setter	set + NomAttribut($val) → setNom($n), setPrix($p), setStock($s)

2. Exemple complet — Classe Produit

Voici la classe Produit avec encapsulation complète : attributs private, constructeur, getters, setter avec validation et méthode métier.

Classe Produit complète — Code PHP annoté

```php
<?php
class Produit {

    private $nom;
    private $prix;
    private $stock;

    public function __construct($nom,$prix,$stock){
        $this->nom   = $nom;
        $this->prix  = $prix;
        $this->stock = $stock;
    }

    public function getNom()  { return $this->nom;  }
    public function getPrix() { return $this->prix; }

    public function setPrix($prix) {
        if ($prix < 0) { echo "Invalide"; return; }
        $this->prix = $prix;
    }

    public function afficher() {
        echo $this->nom." — ".$this->prix."€";
        echo " (stock: ".$this->stock.")";
    }
}

$p1 = new Produit("Stylo", 1.50, 100);
```

- Déclaration de la classe
- private → attributs inaccessibles depuis l'extérieur
- __construct() initialise les 3 attributs en même temps
- Getters : retournent la valeur d'un attribut privé
- Setter avec validation : le prix ne peut pas être négatif
- Méthode métier afficher() accède aux attributs via $this
- Instanciation avec 3 paramètres

À noter : afficher() utilise $this->nom et $this->prix — elle accède directement aux attributs privés car elle EST dans la classe.

Le code externe (hors de la classe) ne peut pas faire $p->nom directement — il passe obligatoirement par getNom().

3. Exercices pratiques

✏ Exercice 1 — Classe Produit — Guidé (30 min)

Contexte :

Reproduisez la classe Produit vue en cours. Testez chaque étape dans votre navigateur avant de passer à la suivante.

Travail demandé :

1. Créez produit.php et déclarez la classe Produit avec les 3 attributs private : $nom, $prix, $stock
2. Ajoutez le constructeur __construct($nom, $prix, $stock) qui initialise les 3 attributs
3. Ajoutez les getters : getNom(), getPrix(), getStock()
4. Ajoutez le setter setPrix($prix) avec validation : refuser les prix négatifs
5. Ajoutez le setter setStock($stock) avec validation : refuser un stock < 0
6. Ajoutez la méthode afficher() qui affiche : "[nom] — [prix]€ (stock : [stock])"
7. Instanciez 2 produits différents et appelez afficher() sur chacun
8. Testez setPrix(-5) — vérifiez que le message d'erreur s'affiche
9. Testez getPrix() et getNom() — vérifiez que les valeurs correctes sont retournées

✏ Exercice 2 — Classe Compte bancaire — Semi-autonome (30 min)

Contexte :

Créez de A à Z une classe CompteBancaire en appliquant les règles d'encapsulation. Un compte a un titulaire, un IBAN et un solde. On peut déposer de l'argent, retirer (si le solde est suffisant) et consulter le solde.

Travail demandé :

10. Déclarez la classe CompteBancaire avec 3 attributs private : $titulaire, $iban, $solde
11. Écrivez le constructeur avec les 3 paramètres (solde initial = 0 par défaut si non fourni)
12. Ajoutez les getters : getTitulaire(), getIban(), getSolde()
13. Ajoutez la méthode deposer($montant) : valider que $montant > 0, puis ajouter au solde
14. Ajoutez la méthode retirer($montant) : valider que $montant > 0 ET que le solde est suffisant, sinon afficher "Solde insuffisant"
15. Ajoutez la méthode afficherInfos() qui affiche toutes les infos du compte
16. Instanciez 2 comptes, effectuez plusieurs dépôts et retraits, vérifiez que les validations fonctionnent